

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	YAZHINI.E.M	Reg.No.:	192511165
Department:	B.E.CSE	Date:	25/07/2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I YAZHINI.E.M /192511165 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Question 1: Signed 8-bit Addition and Subtraction (+45 and -23)

A processor performs arithmetic on signed 8-bit integers using 2's complement representation. Analyze addition and subtraction of +45 and -23, including binary conversion, sign-bit alignment, intermediate steps, and overflow detection.

Answer

Step 1: Binary Conversion (8-bit, 2's Complement)

- +45 = 00101101
- +23 = 00010111 → invert bits = 11101000 → add 1 = 11101001, so -23 = 11101001

Step 2: Addition — (+45) + (-23)

```
  00101101   (+45)
+ 11101001   (-23)
-----
 100010110
```

Discard the carry-out beyond bit 7 (the 9th bit): result = 00010110.

Convert back: 00010110 = 22, sign bit = 0 (positive) → Result = +22 ✓ (matches $45 - 23 = 22$).

Step 3: Overflow Check for Addition

- Carry into sign bit (bit 7) = 1
- Carry out of sign bit = 1
- Since Carry-in = Carry-out → NO overflow

Step 4: Subtraction — (+45) - (-23) = (+45) + (+23)

Subtraction is implemented by adding the 2's complement of the subtrahend. So $-(-23) = +23 = 00010111$.

```
  00101101   (+45)
+ 00010111   (+23)
-----
 01000100
```

Result = 01000100 = +68 ✓ (matches $45 + 23 = 68$).

Step 5: Overflow Check for Subtraction

- Carry into sign bit = 0
- Carry out of sign bit = 0
- Equal → NO overflow (result +68 lies within the 8-bit signed range -128 to +127)

Step 6: How the Processor Detects and Interprets Overflow

The ALU computes overflow using the XOR of the carry into the sign bit and the carry out of the sign bit:

```
Overflow = C(in to MSB) XOR C(out of MSB)
```

Equivalently, overflow occurs only when two operands of the SAME sign produce a result of the OPPOSITE sign. In both operations above the sign combination did not trigger this condition, so both results are valid and correctly represented within the 8-bit signed range.

Question 2: 4-bit Carry Look-Ahead Adder (CLA) vs Ripple Carry Adder

A high-performance system replaces a ripple carry adder (RCA) with a 4-bit CLA to reduce delay. Analyze generate/propagate signal formation, carry computation, and compare speed with justification.

Answer

Step 1: Limitation of the Ripple Carry Adder

Each full-adder stage must wait for the carry produced by the previous stage:

$$C(i+1) = G(i) + P(i) \cdot C(i)$$

Because each carry depends on the previous one, the total delay grows linearly with the number of bits (n), i.e., $\text{delay} \approx n \times (\text{delay per full adder})$.

Step 2: Generate and Propagate Signals

For each bit position i, with inputs A_i and B_i :

- Generate: $G_i = A_i \cdot B_i$ (a carry is generated here regardless of any incoming carry)
- Propagate: $P_i = A_i \text{ XOR } B_i$ (an incoming carry, if present, is passed through this stage)

Step 3: Carry Look-Ahead Computation

Instead of rippling, each carry is expressed directly as a function of the primary inputs (via G and P), so all carries can be computed in parallel:

$$\begin{aligned} C_1 &= G_0 + P_0 \cdot C_0 \\ C_2 &= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0 \\ C_3 &= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0 \\ C_4 &= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0 \end{aligned}$$

Each expression is a two-level sum-of-products (AND-OR) built directly from G and P, which themselves need only one gate delay from A and B. Hence all carries become available almost simultaneously.

Step 4: Sum Output

$$S_i = P_i \text{ XOR } C_i$$

Step 5: Delay Comparison

Adder Type	Delay	Complexity
Ripple Carry Adder	$O(n)$ — grows with bit-width	Simple, less hardware
Carry Look-Ahead Adder	~2–3 gate delays (near-constant for 4 bits)	More gates and wiring, hardware-intensive

Step 6: Why CLA Improves Speed — Justification

- In an RCA, carry propagation is sequential — bit i cannot produce its sum until bit i-1's carry has rippled through.
- In a CLA, every carry is derived directly from the primary inputs A and B through combinational logic, computed in parallel, independent of n.

- This removes the ripple bottleneck, so CLA delay stays nearly constant even as bit-width grows, unlike RCA whose delay scales linearly.
- Modern processors use CLA (often as cascaded 4-bit blocks forming 16/32/64-bit adders) because speed is prioritized over the extra gate/area cost in high-performance ALU design.

Question 3: Booth's Algorithm: Signed Multiplication of -6 and $+5$

Analyze Booth's algorithm step-by-step for -6×5 , including the role of the accumulator, multiplier, and Booth bit, arithmetic shifts, and efficiency versus traditional multiplication.

Answer

Step 1: Representation (5-bit, 2's Complement)

- Multiplicand $M = -6 = 11010$
- Multiplier $Q = +5 = 00101$
- $-M$ (2's complement of M) = 00110
- Registers: Accumulator $A = 00000$, $Q = 00101$, extra Booth bit $Q(-1) = 0$

Step 2: Booth's Bit-Pair Decision Rule ($Q0, Q(-1)$)

$Q0\ Q(-1)$	Operation
0 0	No operation — shift only
0 1	$A = A + M$, then shift
1 0	$A = A - M$ (i.e., $A + (-M)$), then shift
1 1	No operation — shift only

Step 3: Iteration-by-Iteration Execution (5 cycles for a 5-bit multiplier)

Initial: $A = 00000$, $Q = 00101$, $Q(-1) = 0$

Iteration 1

- $Q0\ Q(-1) = 1,0 \rightarrow A = A - M = 00000 - 11010 = 00000 + 00110 = 00110$
- Arithmetic right shift ($A, Q, Q(-1)$): $A = 00011$, $Q = 10010$, $Q(-1) = 1$

Iteration 2

- $Q0\ Q(-1) = 0,1 \rightarrow A = A + M = 00011 + 11010 = 11101$
- Shift: $A = 11110$, $Q = 11001$, $Q(-1) = 0$

Iteration 3

- $Q0\ Q(-1) = 1,0 \rightarrow A = A - M = 11110 + 00110 = 00100$
- Shift: $A = 00010$, $Q = 01100$, $Q(-1) = 1$

Iteration 4

- $Q0\ Q(-1) = 0,1 \rightarrow A = A + M = 00010 + 11010 = 11100$
- Shift: $A = 11110$, $Q = 00110$, $Q(-1) = 0$

Iteration 5

- $Q0\ Q(-1) = 0,0 \rightarrow$ No operation
- Shift: $A = 11111$, $Q = 00011$, $Q(-1) = 0$

Step 4: Final Result

Combine A and Q registers $\rightarrow$ 1111100011 (10-bit 2's complement result).

Converting to decimal: this value equals -30 .

Check: $-6 \times 5 = -30$ ✓

Step 5: Why Booth's Algorithm Is Efficient

- It handles signed numbers directly (positive or negative) without any separate sign-magnitude conversion.
- It reduces the number of add/subtract operations: a run of consecutive 1s or 0s in the multiplier requires only one addition/subtraction at the boundary, instead of one addition per bit as in traditional shift-add multiplication.
- This is especially advantageous when the multiplier contains long runs of 1s or 0s — e.g., a pattern like 0111110 needs only 2 operations instead of 5.

Question 4: Division of Binary Numbers: Restoring vs Non-Restoring ($13 \div 3$)

Analyze restoring and non-restoring division algorithms step-by-step for $13 \div 3$, showing intermediate remainders and quotient formation, then compare the two approaches.

Answer

Step 1: Setup

- Dividend = $13 = 01101$
- Divisor $M = 3 = 0011$
- Registers: Remainder register $A = 0000$, Quotient register $Q = 1101$, $n = 4$ iterations

Step 2: Restoring Division — Method

Procedure per iteration: shift (A, Q) left by 1 bit $\rightarrow A = A - M \rightarrow$ if A is negative, restore by $A = A + M$ and set quotient bit $Q_0 = 0$; otherwise keep A and set $Q_0 = 1$.

Restoring Division — Iteration Trace

Iteration	After Shift	A - M	Decision	Resulting A	Q0
1	A=0001, Q=101_	0001-0011 = 1110 (neg)	Restore	A=0001	0
2	A=0011, Q=010_	0011-0011 = 0000 (≥ 0)	Keep	A=0000	1
3	A=0010, Q=101_	0010-0011 = 1111 (neg)	Restore	A=0010	0
4	A=0101, Q=010_	0101-0011 = 0010 (≥ 0)	Keep	A=0010	1

Final Quotient $Q = 0100 = 4$, Final Remainder $A = 0001 = 1$. Check: $13 = 3 \times 4 + 1$ ✓

Step 3: Non-Restoring Division — Method

- If previous remainder $A \geq 0$: shift left, then $A = A - M$
- If previous remainder $A < 0$: shift left, then $A = A + M$
- Set quotient bit $Q_0 = 1$ if resulting $A \geq 0$, else $Q_0 = 0$
- No restoring step is performed after each subtraction — correction is deferred
- At the very end, if the final A is negative, restore once by adding M back

This means each cycle performs exactly one add or subtract operation, instead of restoring division's occasional extra restore-addition.

Step 4: Comparison of the Two Approaches

Criterion	Restoring Division	Non-Restoring Division
Operations per step	Subtract, plus a possible extra restore-add	Exactly one add or subtract per step
Total operations	Higher (due to extra restores)	Lower and more uniform
Complexity	Conceptually simpler	Slightly more complex control logic
Speed	Slower (conditional restore adds delay)	Faster (single predictable operation)
Hardware	Needs a restore data path	Needs sign-detection logic to choose add/subtract

Step 5: Why Non-Restoring Division Is Preferred

- It performs exactly one addition or subtraction per bit regardless of sign, giving uniform, predictable timing — ideal for pipelined hardware.
- It eliminates the wasted restore cycle present in restoring division, improving throughput.
- Only a single correction is applied at the very end (if needed), rather than after every negative partial remainder, which reduces overall operation count and hardware delay.

Question 5: IEEE 754 Floating-Point Representation of -13.25

Analyze how -13.25 is represented in single and double precision IEEE 754 formats, and explain the floating-point addition process, including normalization, rounding, and precision loss.

Answer

Step 1: Binary Conversion

- 13.25 in binary = 1101.01

Step 2: Normalization

- $1101.01 = 1.10101 \times 2^3$
- Sign: the number is negative $\rightarrow$ Sign bit $S = 1$

Step 3: Single Precision (32-bit: 1 sign + 8 exponent + 23 mantissa)

- Exponent = $3 + \text{bias}(127) = 130 = 10000010$
- Mantissa (23 bits, fraction only) = 10101000000000000000000

```
1  10000010  10101000000000000000000000
```

Step 4: Double Precision (64-bit: 1 sign + 11 exponent + 52 mantissa)

- [illegible]

```
1  100000000010  10101000000000000000000000000000000000000000000000
```

Step 5: Floating-Point Addition Procedure

- Extract fields (sign, exponent, mantissa) from both operands and restore the implicit leading 1 in each mantissa.

- Align exponents: the operand with the smaller exponent has its mantissa right-shifted until both exponents match — this is where bits can be lost.
- Add or subtract the aligned mantissas based on the signs of the operands (differing signs effectively subtract).
- Normalize the result: if the sum overflows (≥ 2.0), shift right and increment the exponent; if leading zeros appear (cancellation), shift left and decrement the exponent.
- Round the mantissa to fit the fixed bit width (round-to-nearest-even is the IEEE default), which may introduce a small rounding error.
- Check the exponent range for overflow (too large $\rightarrow$ infinity) or underflow (too small $\rightarrow$ denormalized number or zero).

Step 6: Key Issues in Floating-Point Addition

- Normalization: required after every operation to keep the standard 1.xxxx form; subtracting nearly equal numbers causes catastrophic cancellation (loss of significant digits).
- Rounding: the mantissa has a fixed bit-width, so results are rounded, introducing small representational errors that can accumulate over repeated operations.
- Precision loss: aligning exponents by shifting the smaller operand's mantissa can permanently discard low-order bits — especially significant when the two operands' magnitudes differ greatly. This is the primary source of floating-point addition inaccuracy.